

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

Д. О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 8

МОДУЛЬ АНАЛИЗА ДАННЫХ В СИСТЕМАХ ИНТЕРНЕТА ВЕЩЕЙ

по курсу:

ИНТЕРНЕТ ВЕЩЕЙ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4326

подпись, дата

Г. С. Томчук

инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Цель работы: приобретение навыков анализа данных в системах интернета вещей.

2 Задание

Создать систему анализа данных, хранящихся в базе. Система должна рассчитывать, как минимум две различных характеристики, например среднее и максимальное значение.

Вариант задания: «Умная теплица».

3 Ход выполнения

В ходе выполнения лабораторной работы была подготовлена система анализа данных, хранящихся в базе данных MongoDB. Было организовано получение записей из коллекции с показаниями датчиков с использованием метода `find`, возвращающего курсор (итерируемую последовательность документов). Далее из каждого документа извлекается поле измеренного значения по ключу `value`, после чего формируется массив числовых значений для последующей обработки. Фрагмент кода, реализующего получение курсора и извлечение значений из записей, изображен на рисунке 1.

```
52     def get_sensor_value_stats(  
53         self,  
54         *,  
55         sensor_type: str,  
56         limit: int = 5000,  
57     ) → Dict[str, Any]:  
58         try:  
59             cursor = (  
60                 self._db["SensorReadings"]  
61                 .find({"sensor.type": sensor_type}, {"value": 1})  
62                 .sort("timestamp", pymongo.DESENDING)  
63                 .limit(int(limit))  
64             )  
65  
66             values: List[float] = []
```

Рисунок 1 — Формирование курсора для поиска значений по датчикам

После получения набора данных был реализован расчет аналитических характеристик. Было добавлено вычисление двух метрик: среднее значение и максимальное значение для выбранного типа датчика. В случае отсутствия данных или возникновения ошибки при обработке было предусмотрено

возвращение пустых значений, чтобы интерфейс мог корректно отобразить отсутствие аналитики. Фрагмента кода с расчетом среднего и максимального значений изображен на рисунке 2.

```
66     values: List[float] = []
67     for doc in cursor:
68         v = doc.get("value")
69         if v is None:
70             continue
71         try:
72             values.append(float(v))
73         except (TypeError, ValueError):
74             continue
75
76     if not values:
77         return {
78             "sensor_type": sensor_type,
79             "count": 0,
80             "avg": None,
81             "max": None,
82         }
83
84     return {
85         "sensor_type": sensor_type,
86         "count": len(values),
87         "avg": float(mean(values)),
88         "max": float(max(values)),
89     }
90 except Exception as exc:
91     return {
92         "sensor_type": sensor_type,
93         "count": 0,
94         "avg": None,
95         "max": None,
96         "error": str(exc),
97     }
```

Рисунок 2 — Расчет среднего и максимального значений

Затем расчет аналитики был встроен в процесс подключения к базе данных: после успешной проверки соединения результаты анализа были добавлены в объект, возвращаемый функцией подключения. Таким образом, вместе со статусом соединения и служебной информацией стало возвращаться поле `analytics`, содержащее рассчитанные характеристики для температурного датчика, датчика влажности воздуха и датчика влажности почвы. Пример итогового объекта ответа на `/connect/database` с добавленным блоком `analytics`

изображен на рисунке 3.



```
1  {
2    "analytics": {
3      "humidity": {
4        "avg": 55.081545064377686,
5        "count": 932,
6        "max": 79.9,
7        "sensor_type": "humidity"
8      },
9      "soil_moisture": {
10       "avg": 40.41223175965665,
11       "count": 932,
12       "max": 60.0,
13       "sensor_type": "soil_moisture"
14     },
15     "temperature": {
16       "avg": 25.07620578778135,
17       "count": 933,
18       "max": 35.0,
19       "sensor_type": "temperature"
20     }
21   },
22   "collections": {
23     "DeviceEvents": 5555,
24     "SensorReadings": 2797
25   },
26   "connection": "ok",
27   "db_name": "greenhouse_logs",
28   "records_total": 8352,
29   "uri": "mongodb://localhost:27017"
30 }
```

Рисунок 3 — Ответ на запрос /connect/database

После реализации вычислений на серверной части была выполнена доработка пользовательского интерфейса. В карточках каждого датчика (температура, влажность воздуха, влажность почвы) были добавлены отдельные поля для отображения статистических данных. Затем была обновлена логика клиентской части, отвечающая за заполнение элементов интерфейса. Было реализовано получение объекта `analytics` из ответа сервера и вывод среднего и максимального значений в соответствующие поля карточек датчиков (рисунок 4).

```

122     const analytics = response.analytics || {};
123     const temp = analytics.temperature || {};
124     const humidity = analytics.humidity || {};
125     const soil_moisture = analytics.soil_moisture || {};
126
127     $("#temperature_avg").val(
128         temp.error ? "ошибка" : formatValue(temp.avg, units.temperature)
129     );
130     $("#temperature_max").val(
131         temp.error ? "ошибка" : formatValue(temp.max, units.temperature)
132     );
133
134     $("#humidity_avg").val(
135         humidity.error ? "ошибка" : formatValue(humidity.avg, units.humidity)
136     );
137     $("#humidity_max").val(
138         humidity.error ? "ошибка" : formatValue(humidity.max, units.humidity)
139     );
140
141     $("#soil_avg").val(
142         soil_moisture.error
143         ? "ошибка"
144         : formatValue(soil_moisture.avg, units.soil_moisture)
145     );
146     $("#soil_max").val(
147         soil_moisture.error
148         ? "ошибка"
149         : formatValue(soil_moisture.max, units.soil_moisture)
150     );
151 },
152 });
153 }
154

```

Рисунок 4 — Разборка объекта analytics и вывод значений

Итоговое отображение рассчитанных значений в карточках датчиков изображено на рисунке 5.

Агроном (оператор)		
Датчик температуры 15.4 °C Среднее 25.0 °C Максимум 35.0 °C	Датчик влажности воздуха 75.3 % Среднее 55.1 % Максимум 79.9 %	Датчик влажности почвы 29 % Среднее 40.4 % Максимум 60.0 %

Рисунок 5 — Обновленный интерфейс. Добавлены поля со статистикой

4 Выводы

В результате выполнения лабораторной работы были получены практические навыки извлечения данных из базы данных в системах Интернета вещей и их последующего анализа. Было реализовано получение

выборки показаний датчиков через курсор MongoDB, преобразование данных к числовому виду и вычисление аналитических характеристик, что позволило перейти от простого накопления измерений к получению обобщающих показателей, пригодных для мониторинга.

Также была продемонстрирована интеграция результатов анализа в прикладной интерфейс. Рассчитанные значения (среднее и максимум) были добавлены в структуру ответа сервера и выведены в пользовательском UI в соответствующих секциях датчиков, что повысило наглядность и удобство контроля состояния системы.

Таким образом, была создана базовая подсистема аналитики для системы умной теплицы, позволяющая автоматически формировать ключевые статистические показатели по накопленным данным и представлять их оператору. Полученный результат может служить основой для дальнейшего развития, включая расширение набора метрик, анализ трендов и построение уведомлений на основе обнаруженных отклонений.

ПРИЛОЖЕНИЕ

Исходный код

Исходный код приложения также доступен по ссылке:

<https://github.com/grigorijtomczuk/smart-greenhouse>.

Листинг 1 — db.py

```
import re
from statistics import mean
from typing import Any, Dict, List, Optional

import pymongo
from werkzeug.wrappers import Request

class Database:
    def __init__(
        self,
        *,
        mongo_uri: str = "mongodb://localhost:27017/",
        db_name: str = "greenhouse_db",
    ):
        self.mongo_uri = mongo_uri
        self.db_name = db_name
        self._client = pymongo.MongoClient(self.mongo_uri)
        self._db = self._client[self.db_name]

    @property
    def db(self):
        return self._db

    def get_status(self) -> Dict[str, Any]:
        try:
            self._client.admin.command("ping")
            collections = list(self._db.list_collection_names())
            counts: Dict[str, int] = {}
            total = 0
            for name in collections:
                c = int(self._db[name].estimated_document_count())
                counts[name] = c
                total += c
            return {
                "connection": "ok",
                "db_name": self.db_name,
                "uri": self.mongo_uri,
                "collections": counts,
                "records_total": total,
            }
        except Exception as exc:
            return {
                "connection": "error",
                "db_name": self.db_name,
                "uri": self.mongo_uri,
                "error": str(exc),
                "collections": {},
                "records_total": 0,
            }
```

```

def get_sensor_value_stats(
    self,
    *,
    sensor_type: str,
    limit: int = 5000,
) -> Dict[str, Any]:
    try:
        cursor = (
            self._db["SensorReadings"]
            .find({"sensor.type": sensor_type}, {"value": 1})
            .sort("timestamp", pymongo.DESCENDING)
            .limit(int(limit))
        )

        values: List[float] = []
        for doc in cursor:
            v = doc.get("value")
            if v is None:
                continue
            try:
                values.append(float(v))
            except (TypeError, ValueError):
                continue

        if not values:
            return {
                "sensor_type": sensor_type,
                "count": 0,
                "avg": None,
                "max": None,
            }

        return {
            "sensor_type": sensor_type,
            "count": len(values),
            "avg": float(mean(values)),
            "max": float(max(values)),
        }
    except Exception as exc:
        return {
            "sensor_type": sensor_type,
            "count": 0,
            "avg": None,
            "max": None,
            "error": str(exc),
        }

def _apply_control(self, request: Request) -> None:
    cmd = request.args.get("database_command", "").strip()
    if not cmd:
        return
    if re.match(r"^[a-zA-Z0-9_\-\-]+$", cmd):
        print(f"[LOG] MongoDB: принята управляющая команда '{cmd}'")
    else:
        print(
            f"[LOG] Ошибка: команда БД '{cmd}' содержит недопустимые символы. "
            f"Разрешены буквы, цифры, '_', '-'"
        )

def connect(self, request: Optional[Request] = None) -> Dict[str, Any]:
    if request is not None:
        self._apply_control(request)

```



```

    status = self.get_status()
    if status.get("connection") == "ok":
        print(
            f"[LOG] MongoDB OK: db={status.get('db_name')}, всего
записей={status.get('records_total')}}"
        )
    else:
        print(f"[LOG] MongoDB ERROR: {status.get('error')}")

    # Аналитика данных
    if status.get("connection") == "ok":
        status["analytics"] = {
            "temperature":
self.get_sensor_value_stats(sensor_type="temperature"),
            "humidity": self.get_sensor_value_stats(sensor_type="humidity"),
            "soil_moisture": self.get_sensor_value_stats(
                sensor_type="soil_moisture"
            ),
        }
    else:
        status["analytics"] = {}

    return status

```

Листинг 2 — index.js

```

...
function getDatabaseData() {
    function formatValue(number, unit, fractionDigits = 1) {
        if (typeof number !== "number" || Number.isNaN(number)) return "-";
        return number.toFixed(fractionDigits) + " " + (unit || "");
    }

    $.ajax({
        type: "GET",
        url: "/connect/database",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            $("#db_connection").val("Соединение: " + response.connection);
            if (response.db_name)
                $("#db_name").val("MongoDB база: " + response.db_name);
            else $("#db_name").val("MongoDB база: -");

            if (typeof response.records_total !== "undefined")
                $("#db_records").val("Записей всего: " + response.records_total);
            else $("#db_records").val("Записей всего: -");

            if (response.collections) {
                const entries = Object.entries(response.collections);
                entries.sort((a, b) => (b[1] || 0) - (a[1] || 0));
                const lines = entries.map(([k, v]) => `${k}: ${v}`);
                $("#db_collections").val(lines.join("\n") || "Коллекций пока нет");
            } else $("#db_collections").val("Коллекций: -");

            const analytics = response.analytics || {};
            const temp = analytics.temperature || {};
            const humidity = analytics.humidity || {};
            const soil_moisture = analytics.soil_moisture || {};

            $("#temperature_avg").val(

```

```

        temp.error ? "ошибка" : formatValue(temp.avg, units.temperature)
    );
    $("#temperature_max").val(
        temp.error ? "ошибка" : formatValue(temp.max, units.temperature)
    );

    $("#humidity_avg").val(
        humidity.error ? "ошибка" : formatValue(humidity.avg, units.humidity)
    );
    $("#humidity_max").val(
        humidity.error ? "ошибка" : formatValue(humidity.max, units.humidity)
    );

    $("#soil_avg").val(
        soil_moisture.error
        ? "ошибка"
        : formatValue(soil_moisture.avg, units.soil_moisture)
    );
    $("#soil_max").val(
        soil_moisture.error
        ? "ошибка"
        : formatValue(soil_moisture.max, units.soil_moisture)
    );
    },
});
}
...

```